

Desenvolvimento de Funcionalidades CRUD (Create e List)

Objetivo da Aula

Compreender o conceito de CRUD e criar serviço REST, com persistência no MySQL via Sequelize, englobando as operações de criação e consulta.

Apresentação

O termo CRUD se tornou amplamente conhecido para os desenvolvedores, dado o perfil natural de sistemas cadastrais. Em termos práticos, é o conjunto mínimo de operações necessário para gerenciar os dados de uma determinada entidade, englobando a criação (Create), leitura (Read), alteração (Update) e exclusão (Delete) dos registros que representam essa entidade.

Nesse contexto, utilizar uma classe DAO, ou Data Access Object, permite que o código fique mais organizado, concentrando as operações sobre o banco de dados. Também é interessante o fato de que a arquitetura REST segue as mesmas operações, permitindo definir uma camada de controle sobre o uso do DAO, com acesso direto a seus métodos. Finalmente, com uma API REST definida, precisamos de um cliente para o acesso aos serviços, e aqui utilizaremos Axios para a comunicação, definindo um sistema cadastral completo, ou seja, a implementação de um CRUD.

1. Conceito de CRUD

Qualquer sistema cadastral deve conseguir recuperar e manipular o conjunto de informações presentes no banco de dados, e como as operações são sempre bastante similares, surge o conceito de **CRUD**, englobando as quatro operações básicas sobre os registros, que são:

- **Create**, referindo-se à inclusão de um registro;
- **Read**, para a recuperação dos dados de uma tabela;
- **Update**, quando o objetivo é alterar os dados atuais do registro; e
- **Delete**, para a remoção do registro.

Um padrão de desenvolvimento muito comum na implementação do CRUD é o **DAO** (Data Access Object), que concentra essas operações em uma classe de gerenciamento para uma entidade específica. Quando o sistema é criado na arquitetura **MVC**, o local correto para a classe DAO é na camada **Model**.

Atualmente, não nos preocupamos com o uso de **SQL**, já que os frameworks de mapeamento **objeto-relacional** fazem a tradução das operações efetuadas sobre as entidades em comandos SQL, bem como mapeiam os resultados das consultas para coleções de entidades. No ambiente do **NodeJS**, o framework **Sequelize** é responsável por essas transformações, e o DAO deve encapsular sua utilização, interpretando as entidades com objetos no formato **JSON**. A tabela seguinte mostra a relação entre as operações do CRUD, os métodos de Sequelize que devem ser utilizados em cada uma delas e o comando SQL que é enviado para o banco de dados.

Quadro 1: Relação entre operações do CRUD, métodos do Sequelize e comandos SQL

Operação	Método	Entrada	Saída	SQL
CREATE	create	entidade	entidade	INSERT
READ	findAll	(não há)	entidade[]	SELECT
	findByPk	identificador	entidade	
UPDATE	update	campos, restrições	quantidade afetada	UPDATE
DELETE	destroy	campos, restrições	quantidade afetada	DELETE

Fonte: Elaboração própria.

O método **create** recebe uma entidade com os dados, persiste no banco e retorna a própria entidade no estado que foi persistido, ou seja, caso você tenha um id auto incrementado, por exemplo, retornará com o id preenchido. Quanto aos métodos **update** e **destroy**, existem diversas formas de utilização, sendo a mais flexível com a passagem dos valores para os campos e as restrições que serão adotadas, retornando com a quantidade de linhas afetadas ao final. Para obter a funcionalidade de um CRUD padrão, as restrições devem ser definidas sobre a chave primária, para que a operação afete apenas um registro.

```
update = async(_id, objJSON) => {
    return await db.Pessoa.update(objJSON,
                                  {where: {id: _id}});
}
```

As restrições também podem ser utilizadas no método **findAll**, permitindo uma consulta parametrizada. Muitas vezes temos métodos de consulta no DAO que são utilizados para fins específicos, como busca por nome ou CPF.

2. Operações Create e List no DAO

Para implementar nosso **DAO**, primeiro, você deve criar um projeto NodeJS com uso de **Sequelize**, definir suas entidades e executar as migrações. Crie um diretório vazio, abra no VS Code, e execute os comandos seguintes.

```
npm init -y
npm install sequelize mysql2
npm install --save-dev sequelize-cli
npx sequelize-cli init
npx sequelize-cli model:generate --name Personagem
    --attributes nome:string,ataque:integer,defesa:integer
```

A sequência de comandos irá configurar o projeto, adicionar as bibliotecas **sequelize** e **mysql2**, além da interface de linha de comando do Sequelize, definir a estrutura necessária para suas entidades e criar a entidade **Personagem**.

Os campos de Personagem são **nome**, **ataque** e **defesa**, onde o primeiro é do tipo **string** e os demais são **integer**. Uma entidade com essa configuração pode ser utilizada na construção de um típico jogo de luta.

Agora, é necessário configurar a conexão com o banco de dados, no arquivo **config.json**, como no fragmento da listagem seguinte. Lembre-se de alterar para a sua senha na configuração.

```
"development": {
  "username": "root", "password": "admin123",
  "database": "jogo_dev", "host": "127.0.0.1",
  "dialect": "mysql"
},
```

Já podemos criar nosso banco de dados e a tabela para a entidade, através dos comandos da listagem seguinte.

```
npx sequelize-cli db:create  
npx sequelize-cli db:migrate
```

Como podemos observar, a interface de linha de comando do Sequelize, via comando **db:create**, gerou o banco de dados **jogo_dev**, enquanto **db:migrate** executou a **migração**, criando a tabela de personagens no banco.

Tendo configurado o ambiente de persistência, podemos iniciar a definição de nosso CRUD, com as operações de **inserção** e **consulta**. Para deixar nosso código organizado, utilizaremos o padrão **DAO**, concentrando as chamadas para o framework Sequelize e permitindo que o restante do sistema trabalhe apenas com objetos do tipo **Personagem**.

Agora crie a pasta **daos** e dentro dela o arquivo **personagem-dao.js**, onde será utilizado o código da listagem seguinte.

```
const db = require('../models/index.js');  
  
class PersonagemDAO{  
  create = async(objJSON) => {  
    return await db.Personagem.create(objJSON);  
  }  
}  
  
module.exports = PersonagemDAO
```

Iniciamos a construção de nossa classe **PersonagemDAO**, com a definição do método **create**, que deverá receber os campos **nome**, **ataque** e **defesa** do personagem no formato **JSON**. Será invocado o método **create**, do **Sequelize**, retornando uma entidade completa, após receber o **id** auto incrementado. Dando continuidade à implementação do DAO, acrescente os métodos que são apresentados na listagem seguinte, referentes à consulta.

```
const db = require('../models/index.js');

class PersonagemDAO{
  // Outros métodos permanecem inalterados

  findAll = async() => {
    return await db.Personagem.findAll();
  }
  find = async(id) => {
    return await db.Personagem.findByPk(id);
  }
}

module.exports = PersonagemDAO
```

Temos os métodos de consulta **findAll**, que retorna todas as entidades que estão no banco de dados, através do método de mesmo nome no **Sequelize**, e **find**, que retorna um personagem a partir de seu **id**, sendo aqui utilizada uma chamada para **findByPk**. Esses são os métodos de consulta obrigatórios para um CRUD, mas nada impede que tenhamos outros tipos de consultas no DAO.

Com a inclusão e consulta resolvidos ao nível do DAO, podemos iniciar a criação de uma interface de serviços. Na prática, concluímos a implementação da camada **Model**, e vamos iniciar a construção da camada **Controller**, dentro de uma arquitetura MVC.

3. Consulta e Inserção via REST

Vamos configurar um servidor Express, mas que agora será acessado a partir de outro domínio, exigindo o uso de **CORS**, para evitar o bloqueio. Inicialmente adicione as bibliotecas necessárias, executando os comandos seguintes.

```
npm install express
npm install body-parser
npm install cors
```

Em seguida, crie o diretório **rotas**, e dentro dele o arquivo **jogo-rotas.js**, com a utilização do código apresentado na listagem seguinte.

```
const express = require('express')
const PersonagemDAO = require('../daos/personagem-dao.js')
const router = express.Router();
const dao = new PersonagemDAO();

router.get('/', async(req, res)=>{
    let dados = await dao.findAll();
    res.json(dados);
})
router.get('/:id', async(req, res)=>{
    let dados = await dao.find(req.params.id);
    res.json(dados);
})
router.post('/', async(req, res)=>{
    let dados = await dao.create(req.body);
    res.json(dados);
})
module.exports = router
```

Foi definido um **endpoint** sobre a **raiz**, que responde com o retorno de todas as entidades para o método **GET**, ou acrescenta uma entidade para o **POST**. Em ambos os casos é utilizado o **DAO**, e no caso da inserção, os dados são recuperados do **corpo** da requisição via **req.body**.

Temos ainda um **endpoint parametrizado**, que recebe o **id** a partir da **rota**, respondendo ao método **GET** com a entidade que corresponde ao id fornecido. Novamente, o **DAO** é utilizado, e o **id** foi recuperado via **req.params.id**, para que fosse utilizado no método de busca **find**.

É importante observar que estamos respeitando estritamente o padrão da arquitetura **REST**, onde **GET** representa **consulta** e **POST** uma **inserção**. Outro ponto a ser observado é a extensa utilização de **async** e **await**, o que se justifica pela natureza **assíncrona** das

operações. Agora, vamos definir nosso servidor, no arquivo **index.js**, com o conteúdo da listagem apresentada a seguir.

```
const express = require('express');
const bodyParser = require('body-parser');
const cors = require('cors')
const rotaJogo = require('./rotas/jogo-rotas');
const app = express();
app.use(bodyParser.json());
app.use(cors({origin: '*' }));
app.use('/personagens', rotaJogo);
app.listen(3000, () => console.log("Servidor pronto"));
```

Como você pode observar no código, foi criado um servidor **Express**, com adoção de **bodyParser** para a **codificação JSON** e uso de **CORS** aberto para **qualquer origem**. Em geral, devemos ter mais cuidado ao diminuir a segurança do CORS, mas aqui estamos apenas viabilizando alguns testes com **Axios** em passos posteriores.

Dando continuidade, associamos o roteador que foi definido anteriormente ao caminho de base **/personagens**, e iniciamos o servidor na porta **3000**. Para executar o servidor, deve ser utilizado o comando **node index**, e o acesso aos serviços ocorre a partir do endereço **http://localhost:3000/personagens**.

Embora seja possível testar a inclusão e a consulta via Postman, Insomnia ou Boomerang, e ainda verificar no MySQL Workbench se os dados na tabela estão corretos, será mais interessante definir uma interface de usuário, ou seja, a camada View da arquitetura MVC.

4. Cliente para Consulta e Inserção com Axios

Visando simular um cenário de aplicação real, vamos trabalhar com outro projeto para o **cliente**, separando o front-end do back-end. Crie um diretório para o novo projeto, abra no VS Code, e utilize os comandos da listagem seguinte.

```
npm init -y
npm install axios
```

A única biblioteca instalada foi a **Axios**, que visa simplificar a construção de requisições HTTP no Java Script. Ela tem várias formas de utilização, mas aqui, como iremos acessar uma API REST, trabalharemos com um objeto Axios que aponta para o endereço de base do Web Service. Crie o cliente de comunicação no arquivo **cliente.js**, utilizando o código que é apresentado a seguir.

```
const axios = require('axios')
const api = axios.create({baseUrl: "http://localhost:3000"});

class Cliente {
  incluir = async(nome,ataque,defesa) => {
    let resp = await api.post('personagens',
                                {nome,ataque,defesa});
    return await resp.data;
  }
  obterTodos = async () => {
    let resp = await api.get('personagens');
    return await resp.data;
  }
  obter = async (id) => {
    let resp = await api.get(`personagens/${id}`);
    return await resp.data;
  }
}
module.exports = Cliente
```

Analisando o código, inicialmente foi definido um objeto **Axios** com o nome **api**, apontado para o endereço de base **http://localhost:3000**. Em seguida, foi definida a classe **Cliente**, que fará a comunicação com o servidor, utilizando o objeto **api**.

No método **incluir**, temos a chamada de **post**, a partir de **api**, fornecendo a **rota** que será utilizada e os **dados do corpo**, no formato **JSON**. Veja como a criação da requisição ficou simples, já com a especificação do método HTTP e o preenchimento dos dados do corpo.

Nos métodos **obterTodos** e **obter**, é invocado o método **get**, a partir de **api**, com o fornecimento da **rota** e sem dados no corpo, já que ambas são operações de consulta. Como a obtenção de um personagem individual requer que o **id** seja fornecido **na rota**, utilizamos o **texto dinâmico** do Javascript para montar a rota de forma adequada.

Em qualquer situação, a variável **resp** recebe a resposta do servidor. Por se tratar de **processo assíncrono**, o retorno de seu conteúdo, no atributo **data**, requer o uso de **await**.

O cliente de comunicação está completo, por hora, e já podemos definir a interface de usuário. Aqui, utilizaremos a interface de **linha de comando**, onde o console já efetua a saída para vídeo, mas precisaremos encapsular a leitura do teclado e definir a lógica geral do aplicativo. Crie o arquivo **index.js**, no novo projeto, e utilize a seguinte codificação inicial, onde temos a lógica geral da interface de usuário.

```
const readline = require('readline');
const Cliente = require("./cliente.js")
const cliente = new Cliente();
const leitor = readline.createInterface({
    input: process.stdin, output: process.stdout
});

const ler = (pergunta) => {
    return new Promise(resolve => {
        leitor.question(pergunta, (input) => resolve(input))
    });
}

const menu = async () => {
    console.log("1-Listar 2-Incluir 3-Alterar 4-Excluir 0-Sair");
    const valor = await ler("Digite a opcao:");
    return eval(valor);
}
```

```
const incluir = async () => { }
const alterar = async () => { }
const excluir = async () => { }
const listar = async () => { }

const principal = async () => {
    let opcao;
    do {
        opcao = await menu();
        switch(opcao){
            case 1: await listar(); break;
            case 2: await incluir(); break;
            case 3: await alterar(); break;
            case 4: await excluir(); break;
        }
    } while (opcao != 0);
}

principal().then(()=>leitor.close());
```

Inicialmente, definimos uma instância para o **cliente de comunicação** e outra para o **leitor de teclado**, gerado a partir do método **createInterface**. No passo seguinte, definimos uma função **ler**, que recebe uma **pergunta** e utiliza o leitor de teclado para mostrar essa pergunta e captura a resposta digitada, através do método **question**. Observe o uso de **Promise** para o retorno assíncrono.

**Destaque**

Devido à natureza das operações de I/O, todas as funções que aqui foram implementadas são assíncronas.

A função **menu** apresenta na tela as opções do sistema e solicita ao usuário que digite a opção desejada. Ela é utilizada pela função **principal** que, dentro de um loop **do-while**, invoca o menu e inicia a execução da função relacionada ao número digitado, saindo do loop quando é digitado o valor 0.

Observado a estrutura switch-case, temos as seguintes opções:

- Listar;
- Incluir;
- Alterar; e
- Excluir.

Em termos práticos, nossa interface está funcional, apresentando o menu e invocando os métodos, mas ainda não temos as operações concretas. Vamos iniciar pela consulta aos personagens, modificando o método **listar** para o que é apresentado no fragmento de código seguinte.

```
const listar = async () => {  
    const dados = await cliente.obterTodos();  
    for(let obj of dados)  
        console.log(obj);  
}
```

Tudo que a função faz é utilizar o **cliente de comunicação** para recuperar todos os personagens, aguardando a resposta com **await**, e imprimir os dados no **console**. Como sabemos que será retornado um vetor, podemos utilizar uma estrutura for para impressão de registros individuais.

Neste ponto já seria possível utilizar a **consulta**, mas vamos acrescentar o tratamento da inclusão. Acrescente a função **obterDados** e modifique a função **incluir**, utilizando o fragmento de código da listagem seguinte.

```
const obterDados = async() => {  
    const nomeStr = await ler("Nome:");
```

```
const ataqueStr = await ler("Ataque:");
const defesaStr = await ler("Defesa:");
return [nomeStr, eval(ataqueStr), eval(defesaStr)];
}

const incluir = async () => {
  const [nome, ataque, defesa] = await obterDados();
  await cliente.incluir(nome, ataque, defesa);
}
```

No método **obterDados**, capturamos os valores digitados pelo usuário, e, no final, retornamos na forma de **vetor**, com a conversão para número feita por **eval**. Os valores são utilizados na função **incluir** para alimentar as variáveis **nome**, **ataque** e **defesa**, utilizadas na sequência para efetuar a inclusão por meio do cliente de comunicação.



Destaque

Ao final do código de **index.js**, acionamos a função **principal** e, por meio da cláusula **then**, fechamos o **leitor** quando terminada a execução.

Com tudo configurado, execute os dois projetos, primeiro o servidor e depois o cliente, ambos utilizando o comando **node index**, e explore as funcionalidades de inclusão e listagem de personagens da interface cliente.

Figura 1: Execução do cliente de linha de comando

```
nome: 'Dragao',
ataque: 10000,
defesa: 10000,
createdAt: '2023-08-29T22:11:39.000Z',
updatedAt: '2023-08-29T22:11:39.000Z'
}
1-Listar 2-Incluir 3-Alterar 4-Excluir 0-Sair
Digite a opcao:
```

Fonte: Elaboração própria.

Considerações Finais da Aula

Após conhecer o conceito de CRUD e compreender a importância do padrão DAO, utilizamos o Sequelize para efetuar as operações sobre o banco de dados, bem como o mapeamento objeto-relacional, e encapsulamos as chamadas em uma classe DAO. Isso nos permitiu definir a camada Model de um sistema na arquitetura MVC.

Indo para a camada Controller, utilizamos o Express para a definição de uma API no estilo REST. Devido à presença do DAO, os endpoints funcionaram como simples redirecionadores, recebendo as requisições HTTP, invocando o DAO a partir das informações recebidas, e convertendo os resultados para a resposta HTTP no formato JSON.

Finalmente, utilizamos o Axios para definir uma interface de usuário, e vimos como ele simplifica a utilização dos diferentes métodos do HTTP. Nesse ponto, definimos a camada View do sistema, com as operações de leitura e criação de entidades via chamadas para a API REST.

Material Complementar



Axios – Promise based HTTP client for the browser and node.js

Axios.

O site oficial do Axios oferece todas as informações necessárias para utilização desse ótimo cliente para servidores HTTP.

Link para acesso: <https://axios-http.com/> (acesso em: 14 set. 2023.)

Referências

FLANAGAN, D. **JavaScript: o guia definitivo**. Porto Alegre: Bookman, 2014. ISBN 9788565837484. Disponível em: <https://biblioteca.sophia.com.br/terminal/9564/acervo/detalhe/92164?guid=1691771378051&returnUrl=%2fterminal%2f9564%2fresultado%2flistar%3fguid%3d1691771378051%26quantidade-Paginas%3d1%26codigoRegistro%3d92164%2392164&i=8>. Acesso em: 14 set. 2023.

OLIVEIRA, Cláudio Luís V.; ZANETTI, Humberto Augusto P. **JavaScript descomplicado: programação para a Web, IoT e dispositivos móveis**. São Paulo: Érica, 2020. ISBN 9788536533100. Disponível em: <https://biblioteca.sophia.com.br/terminal/9564/acervo/detalhe/92165?guid=1691773379732&returnUrl=%2fterminal%2f9564%2fresultado%2flistar%3fguid%3d1691773379732%26quantidadePaginas%3d1%26codigoRegistro%3d92165%2392165&i=24>. Acesso em: 14 set. 2023.

OLIVEIRA, Cláudio Luís V.; ZANETTI, Humberto Augusto P. **Node.js: programe de forma rápida e prática**. São Paulo: Expressa, 2021. ISBN 9786558110217. Disponível em: <https://biblioteca.sophia.com.br/terminal/9564/acervo/detalhe/92479?guid=1691770593236&returnUrl=%2fterminal%2f9564%2fresultado%2flistar%3fguid%3d1691770593236%26quantidadePaginas%3d1%26codigoRegistro%3d92479%2392479&i=4>. Acesso em: 14 set. 2023.